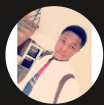




Automate Testing with GitHub Actions

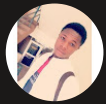


Ahmed Tetteh

The screenshot displays the GitHub Actions interface for a workflow named "RAG CI Pipeline". The workflow is in a failed state, indicated by a red "X" icon and the message "Update k8s.txt to trigger CI workflow #1". The "test-and-build" job is highlighted, showing it failed 1 hour ago in 52s. The job summary lists the following steps and their durations:

- Set up job: 0s
- Checkout code: 1s
- Set up Python: 0s
- Install dependencies: 31s
- Rebuild embeddings: 6s
- Start API in mock mode: 18s
- Run semantic tests: 0s (Failed)

The "Run semantic tests" step is expanded, showing a Python traceback. The error is an `AssertionError` with the message: "Missing 'orchestration' keyword". The traceback indicates the error occurred in `semantic_test.py` at line 15, where an assertion failed: `assert "orchestration" in answer.lower(), "Missing 'orchestration' keyword"`. The final status of the step is "Error: Process completed with exit code 1."



Introducing Today's Project!

In this project, I will demonstrate how to build a CI/CD pipeline with GitHub Actions. I'm doing this project to learn how GitHub can be used beyond just code uploads into repositories, but CI/CD process with GitHub

Key services and concepts

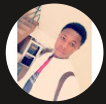
Services I used were GitHub Actions, Ollama, Fast API, Cursor and Python. Key concepts I learnt include how to create and run semantic tests both locally and in the GitHub actions to catch data quality issues and scaling my RAG API to support multiple documents.

Challenges and wins

This project took me approximately 3 hrs (Including study time and research). The most challenging part was learning how to integrate GitHub actions to automate my tests. It was most rewarding to see how CI workflows can be used to catch errors quickly during the build and test phases before they hit production, saving time and resources.

Why I did this project

I did this project because I wanted to learn about GitHub Actions and integrate it with my RAG API. One thing I'll apply from this is the importance of testing - a very crucial part of the CI workflow.

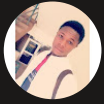


Setting Up Your RAG API

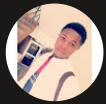
I'm setting up my RAG API by writing an application code and the database required for it, along with installing some dependencies. A RAG API retrieves information by searching the relevant data from a knowledge base, augments a prompt with that retrieved information and Generates a response with an LLM. This foundation is needed for CI/CD because we need to build the RAG API before we can set it up for the GitHub Actions workflow. The RAG API needs to be in working order before and building, testing and deployment can take place.

Local API verification

I tested my RAG API by running `curl -X POST "http://127.0.0.1:8000/query" -G --data-urlencode "q=What is Kubernetes?"` to the /query endpoint. The API responded with an AI generated response. This confirms that my FastAPI's endpoints work well and my prompts + retrieved data are used by the LLM to generate the response for me.



```
• (.venv) king@king-TF65577XG:~/Desktop/nextwork-rag-api$ curl -X POST "http://127.0.0.1:8000/query"
-G --data-urlencode "q=What is Kubernetes?"
{"answer": "Response: Kubernetes, or Kubernetes, is a container orchestration platform that enables
organizations to manage containers at scale. It allows for the deployment, management, and monito
ring of multiple types of containers (e.g., Docker swarm, Kubernetes cluster) on a single platform"}
```

Initializing Git and Pushing to GitHub

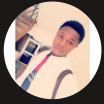
I'm initializing Git for the project using "git init". Git tracks changes by. made to your code over time. Version control enables CI/CD to track changes to my code and revert back to earlier versions if needed.

Git initialization and first commit

I initialized Git by using the "git init" command on my project directory. Then, I staged and committed with "git add ." and "git commit -m "Initial commit: set up local RAG API project" ". The .gitignore file helps by taking out files that I do not want tracked or saved in the git version history. This keeps my repository clean and faster to clone.

Pushing to GitHub for CI/CD

Pushing to GitHub means uploading all your code or project files into a repository. This enables CI/CD because GitHub comes with GitHub actions, i.e, a CI/CD tool baked into it. So, once a push is made, changes are detected, tests are run and deployments are made.



Ahmed Tetteh
NextWork Student

nextwork.org

nextwork-rag-apiPublic

PinWatch0Fork0Star0

mainGo to fileCode

kingswanzy2020Initial commit: set up local RAG API proj...e66d782 · 52 minutes ago

.gitignoreInitial commit: set up local RAG A...52 minutes ago

DockerfileInitial commit: set up local RAG A...52 minutes ago

app.pyInitial commit: set up local RAG A...52 minutes ago

deployment.yamlInitial commit: set up local RAG A...52 minutes ago

embed.pyInitial commit: set up local RAG A...52 minutes ago

k8s.txtInitial commit: set up local RAG A...52 minutes ago

service.yamlInitial commit: set up local RAG A...52 minutes ago

README

Add a README

Help people interested in this repository understand your project.

Add a README

About

RAG API with CI/CD pipeline

Activity0 stars0 watching0 forks

Releases

No releases published

Create a new release

Packages

No packages published

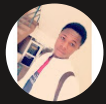
Publish your first package

Languages

Python87.2%
Dockerfile12.8%

Suggested workflows

Based on your tech stack

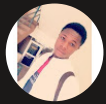


Creating Semantic Tests

I'm creating semantic tests that verify the AI model's response quality. Unlike unit tests that check code logic, semantic tests validate data quality and meaning. This ensures that responses are not just being generated, but they have meaning with reference to the query. These tests ensure quality by helping us understand how the RAG API behaves locally, before its deployment in a pipeline

Non-deterministic output observation

When I ran the query multiple times, I noticed the key word "orchestration" still in the AI generated response, even though I removed it from my knowledge base. This is a problem because my tests become unreliable - as I am unable to tell whether my test worked or not. For CI/CD to work reliably, we need to properly test our application locally, and know its behaviour before automating it in the pipeline - by implementing a mock LLM mode.

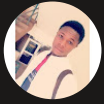


Adding Mock LLM Mode

I'm adding mock LLM mode to validate the retrieval quality of my RAG API, independent of the variably generated responses from the LLM. This solves the non-determinism problem by returning my only the raw context from the embeddings, separate from the variable responses of the LLM. This test proves our retrieval quality, and we have a more deterministic output(same query, same response everytime). Reliable testing requires a mock LLM test to be done to verify your retrieval results and trust your semantic tests.

Mock LLM mode for CI testing

Mock LLM mode returns the retrieved text directly, which makes tests more reliable and deterministic, without giving out any variably generated responses. Without mock mode, tests would be unreliable, as some tests would be True, and others False, due to the randomness of the LLM. For automated CI, we need to verify the retrieval quality of the RAG API during tests, with or without the use of Ollama.

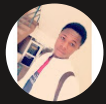


Creating GitHub Actions Workflow

I'm creating a GitHub Actions workflow file that builds, tests and deploys my RAG API automatically directly on the GitHub servers. The workflow automates testing by reading a yaml file uploaded to my repository. The yaml file contains a series of instructions I will define for the CI/CD workflow. When I push code, it will read the yaml file and automatically apply all the steps outlined in the file.

Workflow automation and CI testing

I created the workflow file in ".github/workflows/ci.yml" and pushed it using "git push". Once on GitHub, the workflow will run when GitHub detects changes in these files: k8s.txt, app.py, or embed.py.

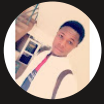


Testing Data Quality

I'm triggering the CI workflow by pushing a change I made in my knowledge base (k8s.txt). The workflow will test for key words in the retrieval results and compare them to the key words found in the semantic_test.py file. I expect it to fail because I intentionally removed the word "orchestration" from the raw context results retrieved from the k8.txt file.

Data quality and CI protection

The missing keyword was "orchestration". The semantic test failed because during the CI process (build & test phase), GitHub detected the keyword was missing from the retrieved context results. This resulted in our CI process to fail, catching the bug early, and giving us the confidence in our responses when LLM is involved. Without CI, this degraded content would have been pushed and deployed to production - the RAG API would have been trained in giving incomplete answers, affecting end-user experience.



← RAG CI Pipeline

Update k8s.txt to trigger CI workflow #1

Re-run jobs

...

Summary

All jobs

test-and-build

Run details

Usage

Workflow file

Annotations

1 error

test-and-build

failed 1 hour ago in 52s

Search logs

...

...

...

...

...

...

Run semantic tests

0s

1 ▶ Run python semantic_test.py

11 Traceback (most recent call last):

12 File "/home/runner/work/nextwork-rag-api/nextwork-rag-api/semantic_test.py", line

22, in <module>

13 test_kubernetes_query()

14 File "/home/runner/work/nextwork-rag-api/nextwork-rag-api/semantic_test.py", line

15, in test_kubernetes_query

16 assert "orchestration" in answer.lower(), "Missing 'orchestration' keyword"

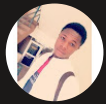
17

18 Error: Process completed with exit code 1.

0s

Success

0s

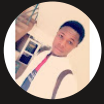


Scaling with Multiple Documents

I'm restructuring the project to handle multiple knowledge base documents. The new folder structure supports multiple documents for the RAG API. This approach scales better because CI can automatically test these documents for to ensure quality response is being generated each time a query is run.

Docs folder structure and CI scaling

The docs folder organizes files by housing all the knowledge base documents needed for my RAG API system. The `embed_docs.py` script handles the embedding all the files in the docs folder - supporting the use of multiple documents and scaling the RAG system to handle questions from multiple knowledge domains. CI validated all documents and found in the docs folder. This structure supports growth by catching issues from multiple documents in the CI workflow before they hit production.



← RAG CI Pipeline

✖

Add NextWork document and update semantic tests for multiple documents #3

Re-run jobs ⌵ ⋮

Summary

All Jobs ⌵

✖

test-and-build

Run details

Usage

Workflow file

> Annotations

1 error

test-and-build

failed 1 minute ago in 43s

Search logs

📄 ↺ ⚙️

> Set up job8s

> Checkout code1s

> Set up Python8s

> Install dependencies22s

> Rebuild embeddings5s

> Start API in mock mode10s

⌵

✖

Run semantic tests1s

1 ▶ Run python semantic_test.py

11 Traceback (most recent call last):

12 ✓ Kubernetes query test passed

13 File "/home/runner/work/nextwork-rag-api/nextwork-rag-api/semantic_test.py", line 37, in <module>

14 test_nextwork_query()

15 File "/home/runner/work/nextwork-rag-api/nextwork-rag-api/semantic_test.py", line 30, in test_nextwork_query

16 assert "maximus" in answer.lower(), "Missing 'maximus' keyword"

17

18 AssertionError: Missing 'maximus' keyword

19 Error: Process completed with exit code 1.



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

